

**VISVESVARAYA TECHNOLOGICAL UNIVERSITY**  
**JnanaSangama, Belagavi– 590018**



**Project Report**  
**On**  
**“INTRUSION DETECTION SYSTEM WITH MACHINE**  
**LEARNING”**

*Submitted in partial fulfillment of the requirement for the award of the degree of*

**BACHELOR OF ENGINEERING**  
**In**  
**CSE (IoT & Cyber Security with Blockchain Technology)**

**By**

|                        |                   |
|------------------------|-------------------|
| <b>AARON QUADRAS</b>   | <b>4MT22IC001</b> |
| <b>ADVAITH M JOY</b>   | <b>4MT22IC005</b> |
| <b>SHAKTHIPRASAD</b>   | <b>4MT22IC049</b> |
| <b>SHREYAS SHENAVA</b> | <b>4MT22IC052</b> |

**Under the Guidance of**  
**Dr.Shivananda V Seeri**  
**Head Of Department**



**DEPARTMENT OF CSE (IoT & Cyber Security with Blockchain Technology)**

**MANGALORE INSTITUTE OF TECHNOLOGY &ENGINEERING**  
(A Unit of Rajalaxmi Education Trust ®, Mangalore)  
**Autonomous Institute Affiliated to V.T.U, Belagavi , Approved by AICTE, New Delhi**  
Accredited by NAAC with A+ Grade & ISO 9001:2015 Certified Institution  
**2025-2026**

MANGALORE INSTITUTE OF TECHNOLOGY & ENGINEERING

(A Unit of Rajalaxmi Education Trust ®, Mangalore)  
Autonomous Institute Affiliated to V.T.U, Belagavi , Approved by AICTE, New Delhi  
Accredited by NAAC with A+ Grade & ISO 9001:2015 Certified Institution



DEPARTMENT OF CSE (IoT & Cyber Security with Blockchain Technology)

CERTIFICATE

This is to certify that Mr. Aaron Quadras (4MT22IC001), Advait M Joy (4MT22IC005), Shakthiprasad Shenoy (4MT22IC049), Shreyas Shenava (4MT22IC052) has satisfactorily completed the project entitled “INTRUSION DETECTION SYSTEM WITH MACHINE LEARNING” for the Major Project Work (BIC786) as prescribed by the VTU for 7<sup>th</sup> semester B.E. CSE (IoT & Cyber Security with Blockchain Technology) branch for the academic year 2025 – 2026. It is certified that all the corrections or suggestions indicated for internal assessment have been incorporated in the report and is been verified and validated.

.....  
Guide Name  
Dr. Shivananda V. Seeri

.....  
Dr. Shivananda V. Seeri  
Head of the Department

.....  
Dr. Prashanth C.M  
Principal

External Viva

Name of Examiners

Signature of the Examiners

1. ....

.....

2. ....

.....

# MANGALORE INSTITUTE OF TECHNOLOGY & ENGINEERING

(A Unit of Rajalaxmi Education Trust ®, Mangalore)

Autonomous Institute Affiliated to V.T.U, Belagavi, Approved by AICTE, New Delhi

Accredited by NAAC with A+ Grade & ISO 9001:2015 Certified Institution

## DEPARTMENT OF CSE (IoT & Cyber Security with Blockchain Technology)



## DECLARATION

We, Mr. **Aaron Quadras** (4MT22IC001), **Advaith M Joy** (4MT22IC005), **Shakthiprasad Shenoy** (4MT22iC049) and **Shreyas Shenava** (4MT22IC052) student of 7<sup>th</sup> semester BE in CSE (IoT & Cyber Security with Blockchain Technology), **Mangalore Institute of Technology and Engineering, Moodabidri**, hereby declare that the project work entitled **“INTRUSION DETECTION SYSTEM WITH MACHINE LEARNING”**, submitted to the **Visvesvaraya Technological University, Belagavi** during the academic year **2025-26**, is a record of an original work done by us under the guidance of **GUIDE NAME, DESIGNATION**, in the Department of CSE (IoT & Cyber Security with Blockchain Technology), Mangalore Institute of Technology and Engineering Moodabidri. This project work is submitted in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Computer Science & Engineering. The results embodied in this report have not been submitted to any other University or Institute for the award of any degree.

Date:

Place:

**AARON QUADRAS**  
**ADVAITH M JOY**  
**SHAKTHIPRASAD**  
**SHREYAS SHENAVA**

# ABSTRACT

Intrusion Detection Systems (IDS) play a crucial role in safeguarding modern computer networks by identifying malicious activities and unauthorized access attempts. As cyberattacks continue to grow in complexity, traditional signature-based systems fail to detect new or evolving threats. To address this challenge, we propose an automated IDS solution that employs Machine Learning techniques to distinguish normal network behavior from intrusive actions. In this work, we use an efficient ML model capable of learning meaningful patterns from network traffic features such as packet size, protocol type, connection duration, and flow behavior. The model is trained and tested on benchmark intrusion datasets to classify traffic as either benign or malicious. Experimental results demonstrate high detection accuracy and strong generalization across various attack types, indicating the reliability of the system. The presented IDS can function as a real-time monitoring tool, a research instrument, and a security layer for organizations, thereby reducing the risk of breaches, enhancing situational awareness, and contributing to overall network safety.

# ACKNOWLEDGEMENTS

The satisfaction and the successful completion of this project would be incomplete without the mention of the people who made it possible, whose constant guidance encouragement crowned our efforts with success.

This project is made under the guidance of **Dr. Shivananda V. Seeri, Professor and Head**, in the Department of CSE (IoT & Cyber Security with Blockchain Technology). We would like to express our sincere gratitude to our guide for all the helping hand and guidance in this project.

We would like to express appreciation to **Dr. Shivananda V. Seeri, Professor and Head**, Department of CSE (IoT & Cyber Security with Blockchain Technology), for his support and guidance.

We would like to thank our Principal **Dr. Prashanth C M**, for encouraging us and giving us an opportunity to accomplish the project.

We also thank our management who helped us directly or indirectly in the completion of this project. Our special thanks to faculty members and others for their constant help and support.

Above all, we extend our sincere gratitude to our parents and friends for their constant encouragement with moral support.

**Aaron Quadras**

**Advaith M Joy**

**Shakthiprasad Shenoy**

**Shreyas Shenava**

# TABLE OF CONTENTS

| <b>Contents</b>          | <b>Page No</b> |
|--------------------------|----------------|
| <b>ABSTRACT</b>          | <b>i</b>       |
| <b>ACKNOWLEDGEMENT</b>   | <b>ii</b>      |
| <b>TABLE OF CONTENTS</b> | <b>iii</b>     |
| <b>LIST OF TABLES</b>    | <b>iv</b>      |
| <b>LIST OF FIGURES</b>   | <b>v</b>       |

| <b>Chapter no</b> | <b>TITLE</b>                                |           |
|-------------------|---------------------------------------------|-----------|
| <b>1.</b>         | <b>INTRODUCTION</b>                         | <b>1</b>  |
| 1.1               | Introduction                                | 1         |
| 1.2               | Problem Statement                           | 1         |
| 1.3               | Objectives                                  | 2         |
| 1.4               | Scope of the project                        | 2         |
| <b>2</b>          | <b>LITERATURE SURVEY</b>                    | <b>3</b>  |
| 2.1               | Existing System                             | 3         |
| 2.2               | Limitations of existing systems             | 11        |
| 2.3               | Proposed System                             | 12        |
| <b>3</b>          | <b>SYSTEM REQUIREMENTS SPECIFICATION</b>    | <b>13</b> |
| 3.1               | Overall description                         | 13        |
|                   | 3.1.1 Product Perspective                   | 13        |
|                   | 3.1.2 Product Functions                     | 14        |
|                   | 3.1.3 User classes and Characteristics      | 14        |
|                   | 3.1.4 Design and Implementation constraints | 14        |
|                   | 3.1.5 Assumptions and Dependencies          | 14        |
| 3.2               | Specific Requirements                       | 16        |
|                   | 3.2.1 Hardware Requirements                 | 16        |
|                   | 3.2.2 Software Requirements                 | 16        |
|                   | 3.2.3 Functional Requirements               | 17        |
|                   | 3.2.4 Non-functional Requirements           | 17        |

|          |                                                |           |
|----------|------------------------------------------------|-----------|
| <b>4</b> | <b>SYSTEM DESIGN</b>                           | <b>19</b> |
| 4.1      | Architectural Diagram                          | 19        |
| 4.2      | Class Diagram                                  | 20        |
| 4.3      | Use Case Diagram and Description               | 22        |
| 4.4      | Sequence Diagram                               | 23        |
| 4.5      | Activity Diagram                               | 23        |
| 4.6      | Data flow Diagram                              | 24        |
| <b>5</b> | <b>IMPLEMENTATION</b>                          | <b>26</b> |
| 5.1      | Module implementation                          | 26        |
| 5.1.1    | Data Preparation and Model Training            | 26        |
| 5.1.2    | Dataset Downloading and Preprocessing          | 27        |
| 5.2      | Code Snippet                                   | 27        |
| <b>6</b> | <b>TESTING</b>                                 | <b>32</b> |
| 6.1      | Testing Levels                                 | 32        |
| 6.1.1    | Unit Testing                                   | 32        |
| 6.1.2    | Integration Testing                            | 33        |
| 6.1.3    | System Testing                                 | 33        |
| 6.1.4    | Acceptance Testing                             | 33        |
| 6.2      | Test cases                                     | 34        |
| <b>7</b> | <b>RESULTS AND SNAPSHOTS</b>                   | <b>35</b> |
| 7.1      | Comparative Study of Different Models Accuracy | 35        |
| 7.2      | Front-end Results                              | 36        |
| <b>8</b> | <b>CONCLUSION AND FUTURE WORK</b>              | <b>39</b> |
|          | <b>REFERENCES</b>                              | <b>40</b> |
|          | <b>PAPERS PRESENTED</b>                        |           |

# LIST OF TABLES

| <b>Table No.</b> | <b>Title</b> | <b>Page No.</b> |
|------------------|--------------|-----------------|
| Table 6.2        | Test Cases   | 37              |

# LIST OF FIGURES

| <b>Figure No.</b> | <b>Title</b>                                                | <b>Page No.</b> |
|-------------------|-------------------------------------------------------------|-----------------|
| Figure 4.1        | Architectural Diagram                                       | 23              |
| Figure 4.2        | Class Diagram                                               | 24              |
| Figure 4.3        | Use Case Diagram                                            | 25              |
| Figure 4.4        | Sequence Diagram                                            | 26              |
| Figure 4.5        | Activity Diagram                                            | 27              |
| Figure 4.6        | Data Flow Diagram                                           | 28              |
| Figure 5.1        | Code Snippet for Importing Necessary Datasets and Libraries | 31              |
| Figure 5.2        | Code Snippet for Data Loading and Preprocessing             | 31              |
| Figure 5.3        | Code Snippet for Flask Application for Real time Analysis   | 32              |
| Figure 5.4        | Code Snippet for Monitoring the Network                     | 33              |
| Figure 5.5        | Code Snippet for Dashboard                                  | 34              |
| Figure 7.1        | Interface of Admin Dashboard                                | 39              |
| Figure 7.2        | Interface When Threat is Detected                           | 39              |
| Figure 7.3        | Live Data Capturing                                         | 40              |

# **Chapter – 1**

## **INTRODUCTION**

# CHAPTER 1

## INTRODUCTION

### 1.1 Introduction

Cybersecurity threats are growing more advanced, placing modern networks at serious risk. Traditional rule-based systems struggle to detect fast-changing and unpredictable attacks, so this project develops a machine-learning-based intrusion detection model using the Random Forest algorithm to analyze network traffic and identify malicious activity in real time. Using the NSL-KDD dataset, which contains labeled normal and attack traffic with key features like protocol type, service requests, duration, and byte statistics, the system performs binary classification to distinguish legitimate from harmful connections. A Flask-based web interface allows administrators to input traffic data and obtain instant predictions, enabling efficient monitoring and faster incident response while improving overall network security without requiring deep technical expertise from users.

### 1.2 Problem Statement

Traditional intrusion detection systems struggle to keep pace with evolving cyberattacks, often relying on fixed rules and signatures that fail to detect new or hidden threats. This results in inaccurate alerts, delayed responses, and increased pressure on security teams who depend on timely and reliable information to protect their networks.

To address these challenges, the project aims to develop an intelligent, automated intrusion detection system that uses machine learning to analyze network traffic in real time. By learning complex patterns of normal and malicious behavior, the system delivers accurate, low-noise threat alerts and reduces manual effort. Its goal is to provide a more efficient, adaptive, and user-friendly approach to modern cybersecurity monitoring.

### 1.3 Objectives

The objectives of the project are as follows:

- **Utilize a comprehensive network dataset:** Use the NSL-KDD dataset, which includes features such as protocol type, service requests, connection duration, and traffic statistics, to build a representative model for intrusion detection.
- **Develop a machine learning-based predictive model:** Construct a Random Forest classifier to accurately distinguish between normal and malicious network traffic based on extracted feature patterns.
- **Optimize model performance:** Apply techniques such as feature scaling, feature selection, and hyperparameter tuning (e.g., number of trees, max depth) to maximize detection accuracy and reduce false positives/negatives.
- **Implement a web-based interface:** Create a Flask-based user interface that allows network administrators to input or upload traffic data, receive intrusion predictions, and view alerts in real time through an intuitive dashboard.
- **Ensure model robustness and generalizability:** Validate the model using various test subsets and simulate different attack types to confirm its reliability and adaptability across a range of intrusion scenarios

### 1.4 Scope of the Project

The scope of this project is to develop a user-friendly web-based intrusion detection system capable of accurately identifying normal and malicious network activity using machine learning. The model analyzes network traffic features from the NSL-KDD dataset and classifies them through a trained Random Forest algorithm to detect potential attacks. While the system improves threat recognition and supports security monitoring, it does not replace professional cybersecurity expertise or guarantee absolute accuracy, and is not intended for high-risk, mission-critical environments.

# **Chapter – 2**

## **LITERATURE SURVEY**

## CHAPTER 2

# LITERATURE SURVEY

### 2.1 Existing System

A literature survey or a literature review in a project report is a section that shows the various analysis and research made in the field of our interest and the results already published, considering the various parameters and the extent of the project.

There was plenty of related work and research done in the form of literature survey to acquire the knowledge and skills needed to complete this project. This led us to come across various project work, thesis and technical papers accompanied by various reviews given on them. We will be discussing some of the papers and the work done in them and then reviewing by comparing them to our project work.

**[1] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “Survey of intrusion detection systems: Techniques, datasets and challenges,” *Cybersecurity*, vol. 2, no. 1, 2025. doi: 10.1186/s42400-019-0038-7**

This paper reviews the major techniques, datasets, and challenges in Intrusion Detection Systems (IDS). It categorizes IDS methods into signature-based, anomaly-based, and hybrid approaches, explaining their strengths and weaknesses—signature-based systems detect known attacks well, while anomaly-based ones can identify new threats but with higher false alarms.

The authors also analyze common datasets like KDD99, NSL-KDD, UNSW-NB15, and CICIDS2017, noting that many are outdated or unbalanced, limiting real-world use.

The paper highlights challenges such as real-time detection, data scalability, and reducing false positives, and concludes that machine learning and deep learning can significantly improve IDS performance. It suggests developing hybrid models and realistic datasets for future research to enhance adaptability against evolving cyber threats.

**[2] “A review on machine learning methods for intrusion detection system,” *Applied and Computational Engineering*, vol. 27, no. 1, pp. 57-64, Dec. 2023.**

This paper reviews various machine learning techniques used in Intrusion Detection Systems (IDS) to improve the detection of cyber threats in modern networks. It focuses on how ML can enhance IDS accuracy, adaptability, and efficiency compared to traditional rule-based systems. The study discusses commonly used algorithms such as Decision Trees, Random Forest, Support Vector Machine (SVM), K-Nearest Neighbors (KNN), and Neural Networks, comparing their performance in detecting both known and unknown attacks. It also emphasizes the importance of data preprocessing, feature selection, and balancing datasets to improve model reliability. The paper highlights that while ML-based IDS can detect complex and evolving threats, they often struggle with high computational cost and false positives in real-time scenarios. The authors conclude that hybrid and ensemble approaches, combining multiple ML models, can achieve better accuracy and robustness. Future work includes integrating deep learning and real-time adaptive systems to strengthen IDS effectiveness against emerging cyberattacks.

**[3] G. Kocher and G. Kumar, “Machine learning and deep learning methods for intrusion detection systems: recent developments and challenges,” *Soft Computing*, vol. 25, no. 15, pp. 9731–9763, 2021. doi: 10.1007/s00500-021-05893-0**

This paper explores recent advancements in machine learning (ML) and deep learning (DL) techniques for improving Intrusion Detection Systems (IDS). It provides a detailed comparison of traditional ML algorithms and emerging DL models in detecting modern and complex cyberattacks. The authors discuss popular ML methods such as SVM, Random Forest, and KNN, alongside DL architectures like CNN, RNN, Autoencoders, and LSTM, analyzing their performance in identifying both known and unknown threats. The study highlights that deep learning models can automatically extract high-level features and achieve better detection accuracy, but require large, well-labeled datasets and high computational resources. Key challenges identified include dataset imbalance, real-time processing limitations, and model interpretability. The paper concludes that integrating hybrid ML-DL approaches and using optimized, adaptive models can significantly enhance IDS efficiency and reliability in dynamic network environments

**[4] A. J. A. Immastephy and K. Punitha, “A Systematic Review on Network Intrusion Detection System based on machine learning and deep learning approach,” E3S Web of Conferences, vol. 540, p. 14006, 2024. doi: 10.1051/e3sconf/202454014006**

The This paper presents a systematic review of machine learning (ML) and deep learning (DL) approaches used in Network Intrusion Detection Systems (NIDS). The authors analyze recent research trends, highlighting how these techniques enhance the detection of cyber threats in complex network environments. The study categorizes existing IDS models into supervised, unsupervised, and hybrid learning approaches, comparing their efficiency, accuracy, and adaptability. It emphasizes that deep learning models—such as CNNs, RNNs, and Autoencoders—achieve higher detection rates by automatically learning complex network patterns. However, the paper also identifies challenges like high computational cost, data imbalance, difficulty in feature extraction, and lack of real-time performance. The authors conclude that hybrid ML-DL frameworks, supported by realistic datasets and real-time adaptive learning, can significantly improve intrusion detection accuracy and robustness against evolving cyber threats.

**[5] R. Kimanzi, P. Kimanga, D. Cherori, and P. K. Gikunda, “Deep Learning Algorithms Used in Intrusion Detection Systems - A Review,” arXiv preprint arXiv:2402.17020, 2024. doi: 10.48550/arXiv.2402.17020**

This paper reviews various deep learning algorithms applied in Intrusion Detection Systems (IDS) to enhance network security and detect complex cyberattacks. It focuses on how deep learning models outperform traditional machine learning methods in terms of accuracy, adaptability, and automation of feature extraction. The authors discuss several architectures, including Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), Long Short-Term Memory (LSTM), and Autoencoders, explaining how each contributes to identifying different types of network intrusions. The study highlights that while deep learning significantly improves detection capability, it faces limitations such as high training time, large dataset requirements, and lack of interpretability. The paper concludes that combining multiple deep learning models in hybrid or ensemble frameworks and integrating real-time detection capabilities can further improve IDS performance and reliability in dynamic network environments.

**[6] S. M. Kasongo, “A Deep Learning Technique for Intrusion Detection System**

This paper proposes a deep learning-based Intrusion Detection System (IDS) using a Recurrent Neural Network (RNN) framework to detect network attacks efficiently and accurately. The approach focuses on leveraging RNN’s ability to learn sequential and temporal patterns in network traffic data. The study involves data preprocessing, feature normalization, and training the RNN model using benchmark intrusion detection datasets. The model’s performance was evaluated using metrics like accuracy, precision, recall, and F1-score, showing superior results compared to traditional machine learning methods. The findings indicate that the RNN-based IDS effectively detects both known and unknown attacks, reducing false alarms and improving detection reliability. However, the paper notes challenges such as high computational cost and training time. The author concludes that integrating RNN with other deep learning architectures or optimization algorithms can further enhance IDS performance for real-time and large-scale network environments.

**[7] Y. Fu, Y. Du, Z. Cao, Q. Li, and W. Xiang, “A deep learning model for network intrusion detection with imbalanced data,” Electronics, vol. 11, no. 6, p. 898, 2022. doi: 10.3390/electronics11060898**

This paper presents a deep learning model designed to improve network intrusion detection performance, particularly when dealing with imbalanced datasets—a common issue in cybersecurity research. The authors propose a framework combining data balancing techniques (such as oversampling and data augmentation) with a deep neural network (DNN) to enhance the detection of minority attack classes that are often overlooked by traditional models. The study uses benchmark intrusion detection datasets and evaluates the model using accuracy, precision, recall, and F1-score, showing that the proposed method achieves higher detection rates and reduces false negatives. The results demonstrate that handling data imbalance significantly boosts model performance and generalization. The paper concludes that deep learning with balanced training data can greatly improve IDS reliability and suggests exploring hybrid architectures and real-time implementation for future work.

**[8] A. Belenguer, J. Navaridas, and J. A. Pascual, “A review of Federated Learning in Intrusion Detection Systems for IoT,” arXiv preprint arXiv:2204.12443, 2022. (Submitted to IEEE Internet of Things Journal). doi: 10.48550/arXiv.2204.12443**

This paper reviews the application of Federated Learning (FL) in Intrusion Detection Systems (IDS) for Internet of Things (IoT) environments. It highlights how FL enables collaborative model training across multiple IoT devices without sharing raw data, thereby improving privacy, scalability, and security. The authors discuss how FL-based IDS frameworks combine machine learning and deep learning techniques to detect intrusions while maintaining data confidentiality across distributed networks. The paper examines various FL architectures, communication strategies, and datasets used in IoT intrusion detection. It also identifies key challenges such as non-IID data distribution, limited device resources, communication overhead, and model synchronization issues. The authors conclude that Federated Learning offers a promising direction for building privacy-preserving and decentralized IDS, and recommend further research on lightweight FL models, edge computing integration, and real-time detection mechanisms for large-scale IoT systems.

**[9] P. L. S. Jayalaxmi, R. Saha, G. Kumar, M. Conti, and T. -H. Kim, “Machine and deep learning solutions for intrusion detection and prevention in iots: A survey,” IEEE Access, vol. 10, pp. 121173-121192, 2022. doi: 10.1109/ACCESS.2022.3220622**

This paper provides a comprehensive survey of machine learning (ML) and deep learning (DL) solutions for intrusion detection and prevention in Internet of Things (IoT) networks. It explores how these intelligent approaches can enhance IoT security against diverse and evolving cyber threats. They analyze popular ML and DL models such as SVM, Random Forest, CNN, RNN, and Autoencoders, discussing their effectiveness in improving detection accuracy and automation. The study also emphasizes challenges unique to IoT, including resource constraints, data heterogeneity, scalability, and privacy concerns. The paper concludes that integrating lightweight and energy-efficient ML/DL models with edge and federated learning frameworks can significantly improve IoT intrusion detection performance. Future research directions include adaptive learning systems, real-time response mechanisms, and secure data-sharing techniques for large-scale IoT deployments.

**[10] K. Mahendar and G. Shivakanth, “A Survey of Intrusion Detection Systems Based on Machine Learning for Cloud Security,” SSRG International Journal of Electrical and Electronics Engineering, vol. 12, no. 5, pp. 226-242, 2025. doi: 10.14445/23488379/IJEEE-V12I5P119**

This paper presents a detailed survey of Intrusion Detection Systems (IDS) that use machine learning (ML) techniques to enhance cloud security. It focuses on how ML-based IDS can effectively detect malicious activities in dynamic and large-scale cloud environments. The authors review various ML algorithms—such as Decision Trees, Random Forest, Naïve Bayes, SVM, and Neural Networks—and evaluate their performance in identifying network and application-level attacks within cloud infrastructures. The study discusses key aspects like data preprocessing, feature selection, and real-time threat detection, along with challenges such as data privacy, multi-tenancy, and scalability in cloud systems. The paper concludes that hybrid ML models and cloud-integrated IDS frameworks can significantly improve detection accuracy and adaptability. It suggests future work on deep learning integration, automation using AI, and privacy-preserving mechanisms to strengthen cloud security.

**[11] M. A. Umer, K. N. Junejo, M. T. Jilani, and A. P. Mathur, “Machine learning for intrusion detection in industrial control systems: Applications, challenges, and recommendations,” International Journal of Critical Infrastructure Protection, vol. 38, p. 100516, 2022. doi: 10.1016/j.ijcip.2022.100516**

This paper explores the use of machine learning (ML) for intrusion detection in Industrial Control Systems (ICS), which are vital for managing critical infrastructure like power grids and manufacturing systems. It focuses on how ML can enhance the security and resilience of ICS against sophisticated cyberattacks. The authors review various ML techniques—such as SVM, Decision Trees, Random Forest, and Neural Networks—and discuss their applications in detecting both network-level and process-level intrusions. The study identifies major challenges, including data scarcity, real-time detection constraints, system interpretability, and integration with legacy industrial systems. It also stresses the importance of domain-specific feature engineering and robust dataset creation for effective model training. The paper concludes that combining ML with deep learning and anomaly detection techniques can significantly improve intrusion detection in ICS. The authors recommend future research on

lightweight, interpretable, and adaptive ML models that can operate in real-time industrial environments with minimal disruption.

**[12] M. Mohale and I. C. Obagbuwa, “A systematic review on the integration of explainable artificial intelligence in intrusion detection systems to enhancing transparency and interpretability in cybersecurity,” *Frontiers in Artificial Intelligence*, vol. 7, 2025. doi: 10.3389/frai.2025.1526221**

This paper provides a systematic review on the integration of Explainable Artificial Intelligence (XAI) in Intrusion Detection Systems (IDS) to improve transparency, interpretability, and trust in cybersecurity applications. The authors emphasize that while machine learning (ML) and deep learning (DL) have significantly enhanced IDS accuracy, these models often function as “black boxes”, making it difficult for analysts to understand how decisions are made. The paper reviews various XAI techniques—including LIME, SHAP, saliency maps, and attention mechanisms—that help explain model outputs and make IDS decisions more interpretable. It also identifies key challenges such as trade-offs between accuracy and interpretability, high computational cost, and the lack of standardized frameworks for evaluating explainable IDS. The paper concludes that integrating XAI with IDS can build analyst confidence, support human-in-the-loop security, and enable auditable, transparent AI-driven cybersecurity systems. Future work focuses on developing real-time explainable IDS models and visual interfaces for better understanding of threat detection decisions.

**[13] S. B. Ahamed Maricar, A. Anoop, B. E. Samuel, A. Appukuttan, and K. H. Alsinjlawi, “An Improved Explainable Artificial Intelligence for Intrusion Detection System,” *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 14s, pp. 108–115, 2024.**

This paper proposes an improved Explainable Artificial Intelligence (XAI) framework for enhancing the performance and transparency of Intrusion Detection Systems (IDS). The study focuses on developing an IDS that not only achieves high detection accuracy but also provides clear and interpretable explanations for its decisions. The authors integrate machine learning and deep learning models—such as Random Forest, CNN, and LSTM—with XAI techniques like LIME and SHAP to make the model’s predictions understandable to cybersecurity analysts. The proposed system was tested on benchmark intrusion detection datasets, showing

improved results in terms of accuracy, precision, recall, and interpretability compared to traditional IDS models. The paper concludes that XAI-based IDS enhances analyst trust and decision-making by revealing the reasoning behind detections. Future work includes optimizing the framework for real-time monitoring and scalability in large and dynamic network environments.

**[14] B. Mahbooba, M. Timilsina, R. Sahal, and M. Serrano, “Explainable artificial intelligence (XAI) to enhance trust management in Intrusion Detection systems using decision tree model,” Complexity, vol. 2021, Art. ID 6634811, 11 pages, 2021. doi: 10.1155/2021/6634811**

This paper explores the use of Explainable Artificial Intelligence (XAI) to improve trust and transparency in Intrusion Detection Systems (IDS) by employing a Decision Tree-based model. The authors aim to make IDS decisions more interpretable while maintaining strong detection performance. The study demonstrates how decision trees, inherently interpretable models, can be combined with XAI techniques to help analysts understand the reasoning behind intrusion alerts. The proposed model was evaluated on standard IDS datasets, showing high accuracy and reliability while providing human-understandable explanations for each classification. The paper emphasizes that explainability increases user trust and aids in faster incident response. However, it also notes challenges such as scalability and handling complex attack patterns. The authors conclude that integrating XAI with interpretable ML models like decision trees can enhance trust management in IDS. Future work includes extending the approach to hybrid and real-time intrusion detection frameworks for broader cybersecurity applications.

**[15] Z. A. El Houda, B. Brik, and L. Khoukhi, ““Why should I trust your IDS?”: An explainable deep learning framework for Intrusion Detection systems in internet of things networks,” IEEE Open Journal of the Communications Society, vol. 3, pp. 1164-1176, 2022. doi: 10.1109/ojcoms.2022.3188750**

This paper introduces an explainable deep learning framework for Intrusion Detection Systems (IDS) in Internet of Things (IoT) networks, addressing the critical question, “*Why should I trust your IDS?*”. The study focuses on improving both the accuracy and interpretability of deep learning-based IDS models. The authors design a hybrid deep learning model combining Convolutional Neural Networks (CNN) and Long Short-Term Memory (LSTM) networks for

detecting complex IoT intrusions. To enhance transparency, they integrate Explainable AI (XAI) methods that provide human-readable justifications for each detection outcome. Experiments conducted on benchmark IoT datasets demonstrate that the framework achieves high detection accuracy, while the explainability component helps analysts understand the reasoning behind the model's predictions. The paper concludes that trustworthy and interpretable IDS are essential for secure IoT environments. Future work includes optimizing the framework for real-time intrusion detection, lightweight deployment on IoT devices, and improving interpretability without sacrificing performance.

## **2.2 Limitations of existing systems**

Although numerous research works have investigated machine-learning-based intrusion detection, existing IDS solutions still encounter significant limitations that affect their practical deployment. Many models, including those based on Decision Trees, SVM, KNN, and even deep learning architectures, are trained on limited or outdated datasets such as KDD99, which do not fully represent modern attack patterns. As a result, these systems often struggle to generalize to real-world network environments and fail to detect emerging or zero-day attacks effectively. In several cases, the datasets used are highly imbalanced, with far more normal traffic samples than attack samples, leading to biased predictions and poor performance against minority attack types.

Furthermore, many studies lack proper feature preprocessing and imbalance-handling techniques, which reduces robustness and increases false-positive and false-negative rates. Another key limitation is the absence of real-time processing capabilities; most models are evaluated offline and are not optimized for live network monitoring, delaying intrusion detection. Interpretability also remains a challenge, as many systems provide alerts without explainable reasoning, making it difficult for security personnel to trust and understand predictions. Additionally, user-friendly interfaces and practical deployment frameworks are often overlooked, limiting usability for network administrators.

These gaps highlight the need for a more adaptive, interpretable, and scalable IDS that supports real-time monitoring, handles data imbalance effectively, and provides meaningful insights to assist users in making informed security decisions.

## **2.3 Proposed System**

This project focuses on developing a machine learning-based Intrusion Detection System (IDS) to strengthen network security by accurately detecting and classifying malicious traffic. Traditional rule-based IDS methods often fail to identify new or sophisticated attacks due to reliance on predefined signatures. To overcome these limitations, the system uses supervised learning trained on benchmark datasets such as NSL-KDD and CICIDS2017, which contain labeled instances of normal and malicious network activity. A Random Forest classifier is selected for its reliability, scalability, and ability to handle high-dimensional and imbalanced data. Key network traffic features, including protocol type, connection duration, packet size, flag indicators, and flow characteristics, are used for training. Model performance is evaluated using Accuracy, Precision, Recall, and F1-Score to ensure dependable detection with minimal false alarms.

The system includes a Flask-based web interface that enables users to upload network data or monitor live traffic for real-time intrusion analysis. Detected threats are displayed on an interactive dashboard with alerts and basic visualizations to support quick decision-making. A structured data preprocessing pipeline, including cleaning, normalization, encoding, and oversampling techniques such as SMOTE, ensures high data quality and effective learning. Through this approach, the project addresses key limitations of traditional IDS, offering a scalable, adaptable, and user-friendly solution suitable for modern network environments, with potential for future enhancement through deep learning or hybrid detection models.

**Chapter – 3**

**SYSTEM REQUIREMENT  
SPECIFICATION**

## CHAPTER 3

# SYSTEM REQUIREMENTS AND SPECIFICATION

A Software Requirements Specification (SRS) serves as a comprehensive description of the software system to be developed. It includes both functional and non-functional requirements and often contains a set of use cases that define user interactions essential for the system's performance. Functional requirements are derived from use cases, while non-functional requirements impose design or implementation constraints. Additionally, the SRS outlines the system's hardware requirements, such as memory, processor specifications, and other critical resources necessary to ensure smooth software operation. The objective of the SRS is to rigorously define requirements before the detailed design phase, reducing the need for redesign later.

### 3.1 Overall Description

The development of a machine learning-based Intrusion Detection System (IDS) requires a well-defined system specification to ensure robust, scalable, and accurate implementation. This section outlines both functional and non-functional requirements necessary for building a real-time, efficient IDS that identifies and classifies network intrusions using supervised learning algorithms. Key goals include developing a high-performing predictive model, designing an accessible web interface, and enabling real-time detection for enhanced cybersecurity. The system will leverage pre-processed network traffic data and offer live monitoring capabilities for use in research and enterprise environments.

#### 3.1.1 Product Perspective

The proposed system is designed with modularity, usability, and performance as core priorities, making it suitable for various users and deployment environments. It features a lightweight and user-friendly Flask-based web interface that enables users to upload traffic data, view intrusion predictions, and receive alerts, with dropdown menus and visual graphs to ensure clarity for both technical and non-technical users. The backend employs

a Random Forest or similar supervised machine learning model trained on benchmark datasets such as NSL-KDD and CICIDS2017, with a strong focus on achieving high detection rates while minimizing false positives. To support live security monitoring, the system integrates packet-capture tools such as Scapy or PyShark for real-time data analysis and threat alerts. Additionally, privacy and security are emphasized through a design that avoids storing user data after each session and includes configurable secure logging for auditing purposes.

### **3.1.2 Product Functions**

**Data Preprocessing:** Network traffic data is cleaned, normalized, and encoded to ensure consistency and model compatibility. Categorical features like protocols are one-hot encoded, while scaling is applied to numerical attributes.

**Model Training and Classification:** The Random Forest model is trained to classify traffic as benign or malicious. Hyperparameter tuning is used to maximize detection performance and minimize classification errors.

**Real-Time Threat Detection:** The model processes live or uploaded data and classifies connections in real-time. Alerts are triggered for anomalous traffic.

**Visualization and Alerting:** A web dashboard displays incoming data statistics, predicted classes, and time-series visualizations. Alerts can be sent via email or displayed in-browser.

### **3.1.3 User Classes and Characteristics**

- **Network Security Analysts:** They Use the system for real-time monitoring of intrusions and network traffic trends. Require high model interpretability and accuracy.
- **IT Administrators:** Responsible for system deployment and maintenance. Require system logs, alert configurations, and integration options.
- **Researchers and Students:** Use the system for testing various machine learning models and datasets. Benefit from modular, open architecture.

### 3.1.4 Design and Implementation Constraints

- **Data Availability:** Accessing large, diverse, and up-to-date network traffic datasets that reflect real-world attack patterns is difficult. The project uses benchmark datasets like NSL-KDD and CICIDS2017 and may supplement with simulated traffic to increase coverage of modern threats.
- **Evolving Threat Patterns:** Attack techniques evolve rapidly, and many resemble normal network behavior, making it challenging to maintain consistently high detection accuracy. The system includes regular model retraining and update mechanisms to remain effective against emerging threats.
- **Computational Resources:** Machine learning-based intrusion detection requires substantial processing power for training and real-time classification. Model optimization and efficient use of system resources are applied to ensure smooth performance even on systems with limited computational capacity.
- **Real-Time Processing:** The IDS aims to analyze traffic and generate alerts in near real time. Lightweight model design, efficient preprocessing, and optimized packet-capture tools are essential to deliver timely and actionable threat notifications.

### 3.1.5 Assumptions and Dependencies

#### Assumptions

- The user has access to a secure and stable network environment for testing intrusion detection.
- Users provide valid and authorized network traffic data to ensure accurate classification and ethical system use.
- Users understand basic networking concepts and follow proper operational guidelines when using the IDS interface.

#### Dependencies

- The system depends on reliable machine learning and backend frameworks such as Python, Scikit-Learn, Flask, and packet-capture tools like Scapy

- The effectiveness of intrusion detection depends on the quality, diversity, and relevance of training datasets such as NSL-KDD and CICIDS2017.
- Successful operation requires proper configuration of the web server and network environment to handle data flow, run model inference, and deliver real-time alerts efficiently.

## **3.2 Specific Requirements**

**Data Collection:** System should collect or accept input from packet capture tools or PCAP files, including key network features like source/destination IP, ports, protocols, and flags.

**Preprocessing Pipeline:** Should normalize, encode, and balance traffic datasets, handling class imbalances through techniques such as SMOTE

**Model Training and Evaluation:** Must support supervised model training (Random Forest, SVM, etc.) with evaluation metrics such as Accuracy, Precision, Recall, and F1-Score.

**Web-Based User Interface:** A dashboard must allow users to upload traffic files, monitor live feeds, and review classification results and alerts.

### **3.2.1 Hardware Requirements**

Hardware requirements refer to the physical components needed to operate the web-based system efficiently.

- CPU: Intel Core i5 or higher
- RAM: Minimum 8 GB
- Storage: At least 50 GB for dataset and model files
- GPU: Optional, for faster model training (NVIDIA GTX 1060 or equivalent)

### **3.2.2 Software Requirements**

Software requirements define the tools and technologies essential for developing and deploying the web application.

- Operating System: Windows 11 / Linux / macOS
- Programming Language: Python 3, HTML, CSS, JavaScript.
- Machine Learning Libraries: Scikit-learn, Pandas, NumPy, Flask, Matplotlib, Seaborn, SMOTE (from imbalanced-learn)
- Packet Capture Tools: PyShark or Scapy (for real-time traffic monitoring)
- Integrated Development Environment (IDE): VS Code, Google Colab.

### 3.2.3 Functional Requirements

This section describes the functional requirements of the Mycoscan system, focusing on accurate Threat detection and user interaction.

- **Data Input and Collection:** The system must allow users to upload network traffic files or capture live packet data through the web interface. These inputs serve as the foundation for identifying normal or malicious activity
- **Pre-processing:** Incoming network data must be preprocessed to extract relevant features, normalize values, and format packets for model analysis. This ensures consistent and reliable input for accurate intrusion detection.
- **Feature Extraction and Classification:** Using the Random Forest model, the system must analyze extracted traffic features and classify the data as normal or malicious with high accuracy, detecting potential intrusions across multiple attack types.
- **Real-Time Detection and Alerts:** The IDS must support live traffic monitoring and generate real-time alerts when suspicious activity is detected. Alerts should be delivered through the interface and logging system to support timely security response.
- **Performance Evaluation Metrics:** During testing and updates, the system must compute evaluation metrics such as accuracy, precision, recall, and F1-score to validate and benchmark model performance before deployment.

### 3.2.4 Non-Functional Requirements

Non-functional requirements define the performance, quality, and operational constraints of

the Intrusion detection system

- **Accuracy:** The IDS must detect malicious network activity with high accuracy, ensuring reliable differentiation between normal and attack traffic.
- **Robustness:** The system should maintain stable detection performance across varying network conditions, traffic volumes, and attack types, including unknown or evolving threats.
- **Scalability:** The application should handle increasing network traffic and multiple users without performance degradation, supporting future expansion and larger datasets.
- **Real-time Processing:** The system must analyze traffic and generate intrusion alerts with minimal delay to support timely threat response.
- **Resource Efficiency:** The model and backend must be optimized to operate smoothly on moderate hardware resources while still maintaining high detection performance.
- **Explainability:** The system should provide clear logs or supporting information to help users understand detection results and support debugging or analysis when needed.
- **Privacy and Security:** Network data must be processed securely, with safeguards to prevent unauthorized access, misuse, or logging of sensitive information beyond operational needs.
- **Accessibility:** The web interface should be intuitive, responsive, and accessible across devices, ensuring usability for both technical and non-technical users in practical environments.

# **Chapter – 4**

## **SYSTEM DESIGN**

## CHAPTER 4

### SYSTEM DESIGN

For this project, the goal is to build a practical and dependable Intrusion Detection System that uses machine learning to identify malicious network activity. The system follows a clean, modular structure with a simple web-based frontend and a Flask-powered backend. Users upload network traffic files or trigger live packet capture through the interface, and the backend takes over the analysis process. Once data is received, the system performs preprocessing steps such as feature extraction, encoding, and normalization to make sure the input is ready for accurate evaluation. These processed features are then passed into a Random Forest model, which has been trained on benchmark datasets to recognize patterns associated with normal and malicious traffic. The model then returns a clear classification output, indicating whether the activity is harmless or potentially harmful.

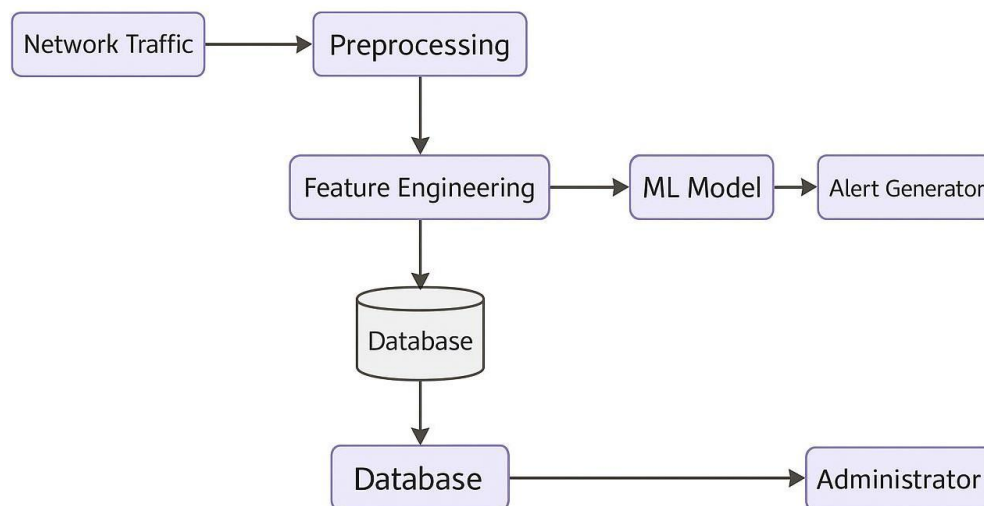
To support real-time monitoring, the system is capable of capturing and analysing live packets through integrated network sniffing tools. The web dashboard displays detection results, along with logs and alert messages, helping users take action quickly when threats are detected. This architecture keeps heavy processing on the backend while ensuring the interface remains responsive and easy to use. The design also supports scalability, making it suitable for future extensions such as deploying deeper learning models, adding alert automation, or integrating with enterprise security systems. Overall, the system is built to offer a practical, efficient, and user-focused approach to intrusion detection, making complex cybersecurity processes easier to manage and understand.

#### 4.1 Architectural Diagram

The architectural diagram illustrates the overall workflow of the system, depicting the interaction among various components involved in the mushroom classification process. The system begins with the user interface, where the user uploads an image of a mushroom through the web application. This image is then sent to the Flask Backend, which acts as the central

processing unit and communication hub between the frontend and the deep learning model. The Flask backend is responsible for handling incoming requests, managing data flow, and initiating the preprocessing and prediction phases. Upon receiving the uploaded image, the backend performs a series of image preprocessing steps, to ensure uniformity and improve model accuracy. The preprocessed image is then passed to the MobileNetV2 model, which extracts critical visual features such as color patterns, texture, and shape to classify the mushroom as edible or poisonous. Additionally, the system can generate a Grad-CAM heatmap that highlights the regions of the image most influential in the classification decision.

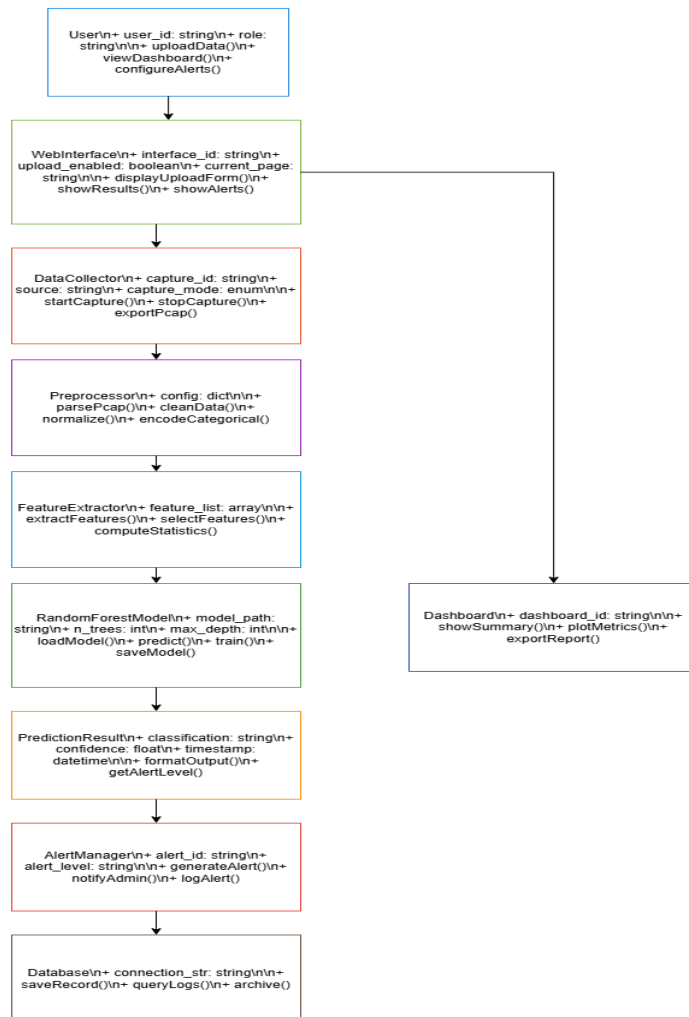
## Intrusion Detection System IDS with ML



**Figure 4.1- Architectural Diagram**

### 4.2 Class Diagram

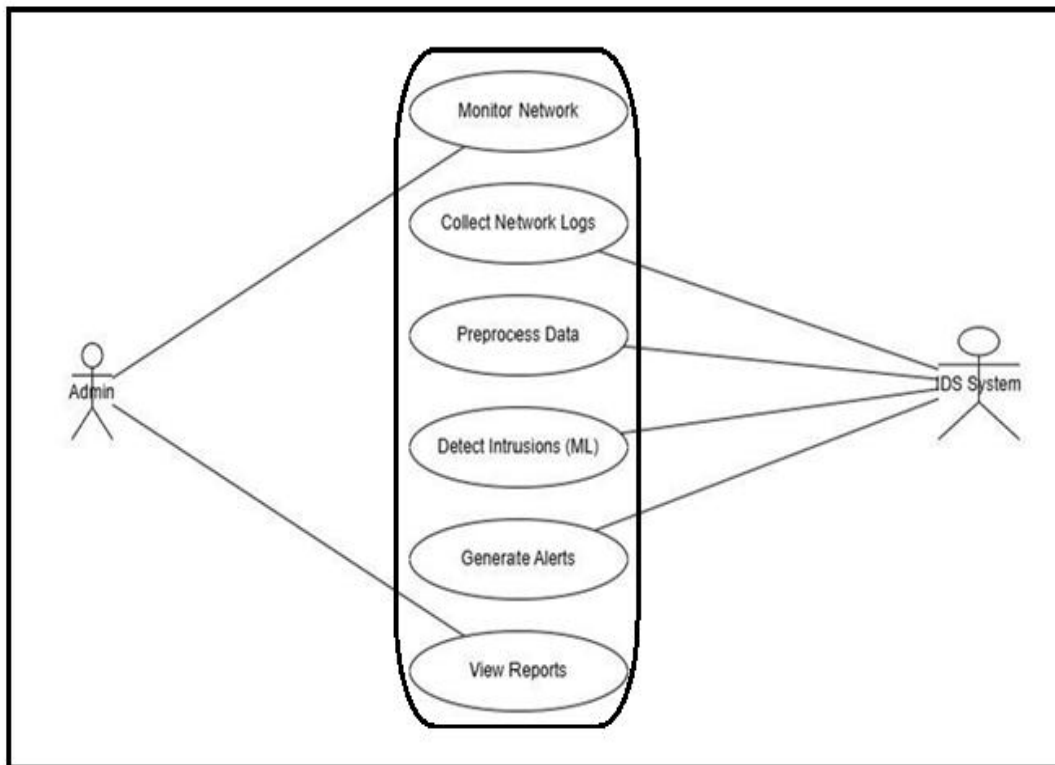
A class diagram is a visual representation of the structure of the system, showing the main classes, their attributes, methods, and how they interact with each other. It illustrates how the user uploads mushroom images through the web interface, which are then processed by the image processor and analyzed by the CNN model (MobileNetV2) to classify mushrooms as edible or poisonous. The diagram also shows supporting classes such as Prediction result, GradCAM generator, and safety guidelines, which handle result display, visualization, and safety tips. This helps in understanding the logical design and flow of the system.



**Figure 4.2- Class Diagram**

### 4.3 Use Case Diagram and Description

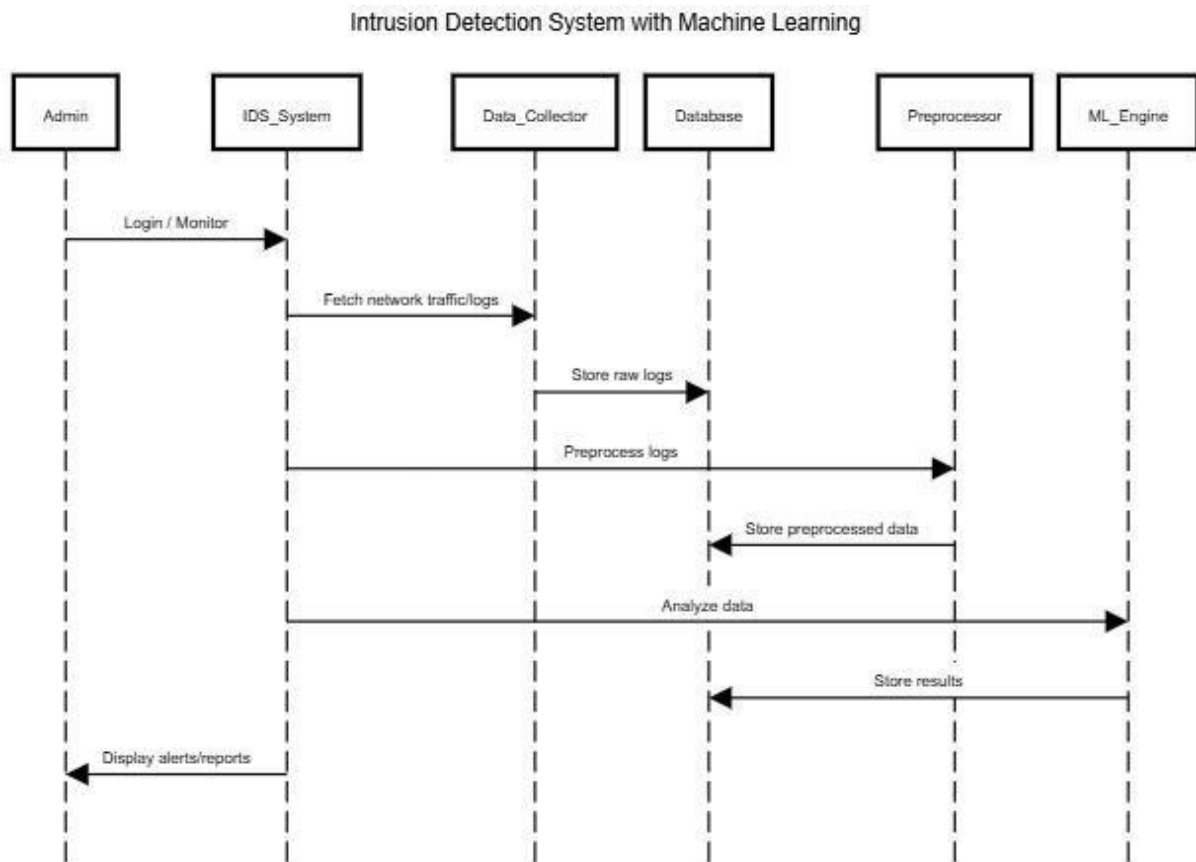
The use case diagram illustrates the interactions between the system and its primary user—the Admin. It outlines the key functionalities that the admin can perform within the IDS system. These include monitoring the network, collecting network logs, preprocessing the collected data, and using machine learning algorithms to detect potential intrusions. The system can also generate alerts based on detected threats and allows the admin to view detailed reports. This structured interaction highlights the system’s capability to automate intrusion detection while providing administrators with actionable insights and comprehensive oversight. The system also supports the system by providing stored images and related information for reference and training. This diagram helps in understanding the functional requirements of the system and gives a clear overview of how users interact with it to perform various operations efficiently.



**Figure 4.3- Use Case Diagram**

## 4.4 Sequence Diagram

The sequence diagram presents a step-by-step interaction between the key components of the Intrusion Detection System (IDS) and the Admin. The process begins when the admin logs in or monitors the system. The IDS system then initiates the data collection process, fetching network traffic logs via the Data Collector. These raw logs are stored in the database and sent to the Preprocessor for cleansing and formatting. Once preprocessed, the data is stored back into the database. The ML Engine then analyzes this processed data to detect potential intrusions. The analysis results are stored and finally displayed to the admin in the form of alerts or reports. This sequential flow highlights the real-time processing capabilities of the system, ensuring timely and accurate intrusion detection Model.

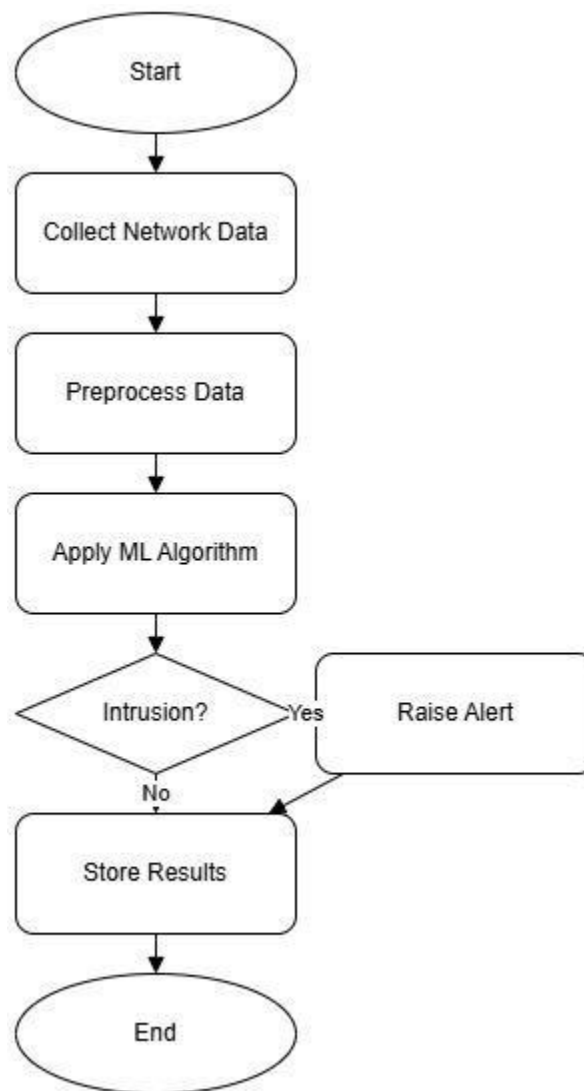


**Figure 4.4- Sequence Diagram**

## 4.5 Activity Diagram

The activity diagram illustrates the end-to-end workflow of the IDS with ML system. It begins

with the admin initiating network monitoring. The system collects network traffic or logs and stores them for processing. In the next stage, the data undergoes preprocessing to clean and format it for analysis. This processed data is then analyzed using machine learning algorithms to detect potential intrusions. If an intrusion is detected, the system generates alerts and stores the results. Finally, these outcomes are presented to the admin through detailed reports or alerts, enabling timely action. The diagram highlights the system's automated, data-driven approach to securing networks.

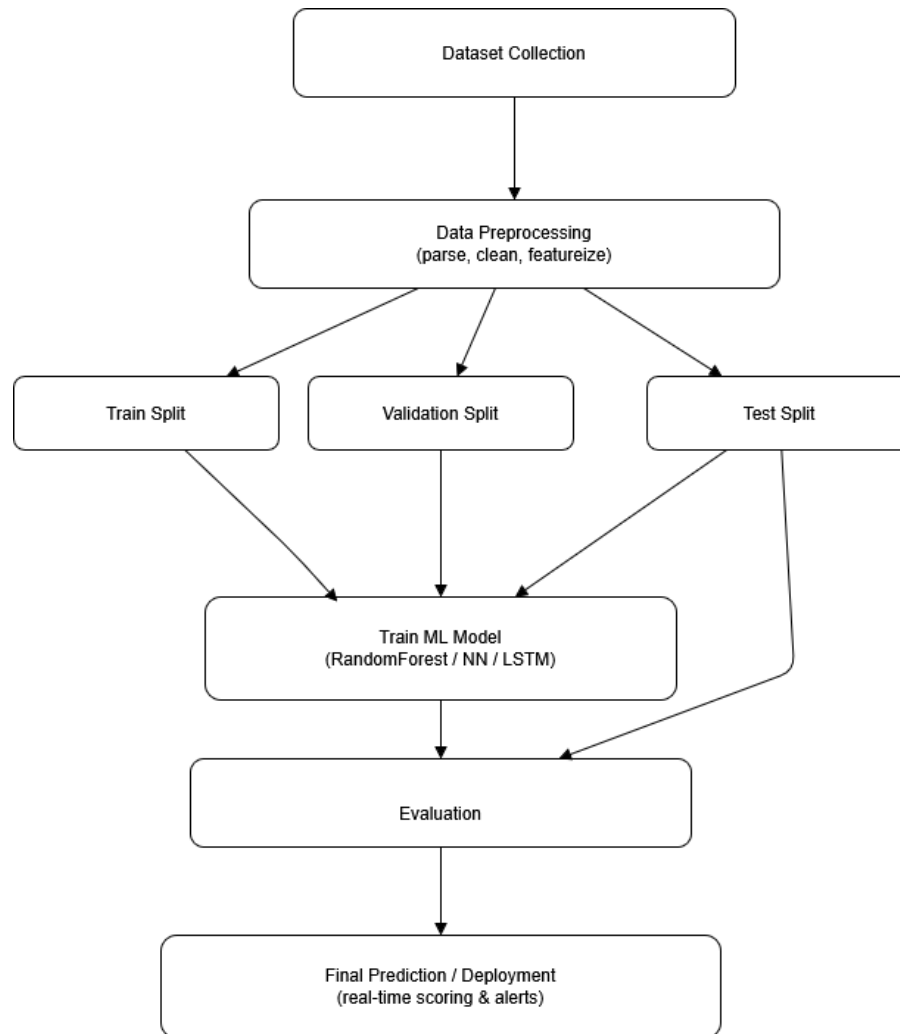


**Figure 4.5- Activity Diagram**

## **4.6 Data Flow Diagram**

A Data Flow Diagram (DFD) is a graphical representation of how data moves through the

system. It illustrates how mushroom images are input, processed, analyzed by the model, and output as classification results (edible or non edible). The DFD also shows interactions between different components such as the frontend interface, backend server, and machine learning model. DFDs in this project help to understand, document, and visualize the flow of data from image upload to prediction and visualization, ensuring a clear overview of the system's working.



**Figure 4.6- Data Flow Diagram**

# **Chapter – 5**

## **IMPLEMENTATION**

## CHAPTER 5

# IMPLEMENTATION

### 5.1 Module Implementation

Implementation refers to the practical realization of the Intrusion Detection System (IDS) for network traffic monitoring using machine learning. This involves converting the planned design into executable code, integrating key components, and ensuring all modules work together for accurate offline (and future real-time) intrusion detection. An implementation plan was formulated prior to the actual coding to address all technical requirements and workflow steps.

#### 5.1.1 Data Preparation and Model Training

Essential Python libraries were used for data handling, machine learning, and evaluation:

The first step in the project implementation involves importing essential Python libraries to handle data, train machine learning models, and evaluate performance.

- Pandas:
  - Used for efficient data manipulation, cleaning, and loading CSV datasets. It provides Data Frame structures that simplify handling tabular data such as the network traffic records used here.
- Scikit-learn modules:
  - `train_test_split` facilitates splitting the dataset into training and testing sets to evaluate how well the model generalizes to unseen data.
  - Random Forest Classifier is the main machine learning algorithm used, chosen for its robustness, interpretability, and ability to handle complex, high-dimensional data.
  - Accuracy score enables evaluation of model predictions by calculating the proportion of correct classifications over total predictions.

- urllib, zipfile, os:
  - Used to download and extract the NSL-KDD dataset automatically if it is not locally available. This automates dataset management and helps ensure reproducibility.
  - By organizing imports at the start, the development environment is prepared and all dependencies loaded for subsequent data processing and model training.

### 5.1.2 Dataset Downloading and Preprocessing

Downloaded the NSL-KDD dataset and prepared the data for model training:

This section automates downloading the NSL-KDD dataset from a public repository. The dataset is extracted and loaded into a Pandas Data Frame for preprocessing.

- Downloading and Extraction: Uses Python's urllib request to download a zip file containing the dataset. The zip file library extracts the required training file. Checking for file existence avoids redundant downloads.
- Loading the Dataset: The data file (KDDTrain+.txt) is read into a Data Frame with predefined column names describing network traffic features such as connection duration, protocol types, service requested, flags, and bytes transferred.
- Data Cleaning: The difficulty column, not required for detection, is dropped to avoid noise and simplify modelling.

## 5.2 Code Snippets

### Code Snippet for Importing Necessary Datasets and Libraries.

```
from scapy.all import sniff, IP, TCP, UDP, ICMP
import pandas as pd
import numpy as np
from collections import defaultdict
import time
from datetime import datetime
import threading
import json
import requests
```

**Figure 5.1-Code Snippet for Importing Necessary Datasets and Libraries**

**Code Snippet for Implementing Datasets**

```
columns = [  
    'duration', 'protocol_type', 'service', 'flag', 'src_bytes', 'dst_bytes',  
    'land', 'wrong_fragment', 'urgent', 'hot', 'num_failed_logins', 'logged_in',  
    'num_compromised', 'root_shell', 'su_attempted', 'num_root', 'num_file_creations',  
    'num_shells', 'num_access_files', 'num_outbound_cmds', 'is_host_login',  
    'is_guest_login', 'count', 'srv_count', 'serror_rate', 'srv_serror_rate',  
    'rerror_rate', 'srv_rerror_rate', 'same_srv_rate', 'diff_srv_rate',  
    'srv_diff_host_rate', 'dst_host_count', 'dst_host_srv_count',  
    'dst_host_same_srv_rate', 'dst_host_diff_srv_rate', 'dst_host_same_src_port_rate',  
    'dst_host_srv_diff_host_rate', 'dst_host_serror_rate', 'dst_host_srv_serror_rate',  
    'dst_host_rerror_rate', 'dst_host_srv_rerror_rate', 'attack_type', 'difficulty'  
]  
  
def load_and_preprocess_data(train_file, test_file):  
    """Load and preprocess NSL-KDD dataset"""  
    print("Loading datasets...")  
  
    train_df = pd.read_csv(train_file, names=columns, header=None)  
    test_df = pd.read_csv(test_file, names=columns, header=None)  
  
    train_df = train_df.drop(['difficulty'], axis=1)  
    test_df = test_df.drop(['difficulty'], axis=1)  
  
    train_df['label'] = train_df['attack_type'].apply(lambda x: 0 if x == 'normal' else 1)  
    test_df['label'] = test_df['attack_type'].apply(lambda x: 0 if x == 'normal' else 1)  
  
    train_df['attack_category'] = train_df['attack_type']  
    test_df['attack_category'] = test_df['attack_type']
```

**Figure 5.2-Code For Data Loading and Preprocessing for NSL-KDD Dataset**

**Code Snippet for Flask Application for Real-Time Analysis**

```
@app.route('/stats', methods=['GET'])
def get_stats():
    """Get statistics"""
    threats = ids_engine.threat_history

    if not threats:
        return jsonify({
            'total_analyzed': 0,
            'threats_detected': 0,
            'threat_rate': 0,
            'avg_confidence': 0
        })

    threats_detected = sum(1 for t in threats if t['threat_detected'])
    confidences = [t['confidence'] for t in threats if t['threat_detected']]

    return jsonify({
        'total_analyzed': len(threats),
        'threats_detected': threats_detected,
        'threat_rate': threats_detected / len(threats) if threats else 0,
        'avg_confidence': np.mean(confidences) if confidences else 0
    })

@app.route('/test-alert', methods=['POST'])
def test_alert():
    """Test Twilio alert"""
    if not ids_engine.twilio_enabled:
        return jsonify({'error': 'Twilio not configured'}), 400

    test_threat = {
        'src_ip': '192.168.1.100',
        'dst_ip': '192.168.1.1',
        'confidence': 0.95,
        'timestamp': datetime.now().isoformat()
    }
```

**Figure 5.3-Code Snippet for Flask Application for Real-Time Analysis**

### Code Snippet for Monitor the Network

```
class NetworkMonitor:
    def __init__(self, interface="eth0", queue_size=100):
        self.interface = interface
        self.queue_size = queue_size
        self.connection_states = defaultdict(lambda: {
            'packets': [],
            'start_time': time.time(),
            'features': {}
        })
        self.feature_window = []
        self.api_url = "http://localhost:5000/analyze"

    def extract_packet_features(self, packet):
        """Extract features from a single packet"""
        try:
            if not packet.haslayer(IP):
                return None

            ip_layer = packet[IP]
            features = {
                'timestamp': datetime.now().isoformat(),
                'src_ip': ip_layer.src,
                'dst_ip': ip_layer.dst,
                'protocol': ip_layer.proto,
                'packet_size': len(packet),
                'ttl': ip_layer.ttl,
            }

            # TCP features
            if packet.haslayer(TCP):
                tcp_layer = packet[TCP]
                features.update({
                    'src_port': tcp_layer.sport,
                    'dst_port': tcp_layer.dport,
                    'tcp_flags': tcp_layer.flags.value if hasattr(tcp_layer.flags, 'value') else 0,
                    'window_size': tcp_layer.window,
                    'protocol_type': 'tcp'
                })
        except Exception as e:
            print(f"Error extracting features: {e}")
```

**Figure 5.4- Code Snippet for Monitor the Network**

### Code Snippet for Dashboard

```

</style>
</head>
<body>
  <div class="container">
    <div class="header">
      <h1>🔒 Intrusion Detection System</h1>
      <p>Real-time Network Threat Monitoring
        <span class="status-indicator status-active" id="statusIndicator"></span>
      </p>
    </div>

    <div class="threat-alert" id="threatAlert">
      <h2>⚠️ THREAT DETECTED!</h2>
      <p id="threatMessage"></p>
    </div>

    <div class="stats-grid">
      <div class="stat-card">
        <h3>Total Analyzed</h3>
        <div class="stat-value" id="totalAnalyzed">0</div>
      </div>
      <div class="stat-card">
        <h3>Threats Detected</h3>
        <div class="stat-value" id="threatsDetected">0</div>
      </div>
      <div class="stat-card">
        <h3>Threat Rate</h3>
        <div class="stat-value" id="threatRate">0%</div>
      </div>
      <div class="stat-card">
        <h3>Avg Confidence</h3>
        <div class="stat-value" id="avgConfidence">0%</div>
      </div>
    </div>

    <div class="charts-grid">
      <div class="chart-container">

```

**Figure 5.5 Code Snippet for Dashboard**

# **Chapter – 6**

## **TESTING**

## CHAPTER 6

# TESTING

Testing is a crucial phase in the software development life cycle. It ensures that the developed system functions according to the specified requirements and performs reliably under various conditions. During this phase, the application was executed with a variety of test cases, and the outputs were compared with the expected results to verify correctness. All errors and inconsistencies identified during testing were rectified, and the corrections were documented for future reference. A series of systematic testing activities were conducted to ensure that the system was stable, accurate, and ready for deployment. It is important to note that software testing is different from Software Quality Assurance (SQA), which focuses on all aspects of quality across the development process, whereas testing focuses solely on defect detection and correction.

### 6.1 Testing Levels

Testing is a crucial part of the Verification and Validation process, ensuring the reliability, functionality, and robustness of the mushroom classification system. The primary objectives of testing are to verify that each component operates as intended, identify and correct any hidden errors in the model or code, and validate that the complete system meets all functional and non-functional requirements. To achieve these goals, testing was performed at multiple levels-Unit Testing, Integration Testing, System Testing, and Acceptance Testing - each focusing on different aspects of the system's performance and overall quality assurance.

#### 6.1.1 Unit Testing

Unit testing for the Intrusion Detection System (IDS) with Machine Learning focused on validating the functionality of individual modules in isolation to ensure each performed as intended. The primary units tested included data preprocessing functions such as feature extraction, normalization, and encoding, along with the machine learning model's training, prediction, and evaluation components.

Specifically, tests were conducted on the Feature Extractor and Preprocessor modules to verify correct transformation of raw network packets and flows into numerical feature vectors. The **Model** unit, which used algorithms such as Random Forest and Neural Networks, was tested

for accurate training behavior, consistent prediction outputs, and correct model loading and saving procedures.

### **6.1.2 Integration Testing**

The Model unit, which implemented algorithms such as Random Forest and Neural Networks, was thoroughly tested to verify correct training behavior, stable convergence, and reproducible performance across multiple datasets. Unit tests also confirmed the reliability of prediction functions, ensuring consistent classification outputs for identical input samples. Furthermore, model persistence functions were verified to ensure accurate saving and loading of trained models without any loss of parameters or degradation in prediction accuracy.

### **6.1.3 System Testing**

System testing was conducted on the fully integrated Intrusion Detection System (IDS) with Machine Learning to evaluate its overall behavior, functionality, and compliance with the specified project requirements. The testing process involved simulating various network traffic scenarios, including both normal and malicious activities such as DoS, Probe, R2L, and U2R attacks. The primary objectives were to verify the system's detection accuracy, response efficiency, and reliability under different network loads while ensuring the proper functioning of all integrated modules, including data preprocessing, feature extraction, machine learning-based detection, rule-based matching, and alert generation.

### **6.1.4 Acceptance Testing**

Acceptance testing confirmed that the final **Intrusion Detection System (IDS) with Machine Learning** met all user and project requirements. The system was evaluated for usability, reliability, and accuracy, successfully detecting network intrusions and generating clear alerts. All acceptance criteria were fulfilled, confirming the IDS was ready for deployment.

## **6.2 Test Cases**

In software engineering, a test case is a specification of the inputs, execution conditions, testing procedure, and expected results that define a single test to be executed to achieve a particular software testing objective, such as to exercise a particular program path or to verify compliance

with a specific requirement. Formally defined test cases allow the same tests to be run repeatedly against successive versions of the software, allowing for effective and consistent regression testing.

Individual PASS/FAIL criteria are written for each test case. All the tests need to get a PASS criterion for proper working of an application

**Table 6.1 Test Cases**

| <b>Test No.</b> | <b>Model Name</b>               | <b>Input</b>                                | <b>Expected Output</b>          | <b>Status</b>                                 | <b>Final</b>       |
|-----------------|---------------------------------|---------------------------------------------|---------------------------------|-----------------------------------------------|--------------------|
| 1               | Random Forest                   | Network packet features (NSL-KDD)           | Classify as Normal or Attack.   | Correct classification on majority samples    | Accepted           |
| 2               | XGBoost                         | Mixed traffic logs (DoS + Normal)           | Detect DoS with high confidence | Slight delay in some detections               | Rejected           |
| 3               | SVM                             | Port-scan feature vector                    | Classify as Scan attack         | low confidence on similar flows               | Partially Accepted |
| 4               | Deep Neural Network             | CIC-IDS2017 HTTP flow                       | Identify Brute Force attack     | Correct detection on test sample              | Accepted           |
| 5               | Unseen zero-day traffic pattern | Malformed packet data                       | Flag as Suspicious              | Misclassified a few normal flows              | Rejected           |
| 6               | Isolation Forest                | Store alert details and logs into database. | Mark as Anomaly                 | Correct anomaly flag but inconsistent scoring | Partially Accepted |
| 7               | KNN                             | Real-time streaming packets                 | Classify per flow               | Very slow under load                          | Rejected           |

# **Chapter – 7**

## **RESULTS AND SNAPSHOTS**

## CHAPTER 7

# RESULTS AND SNAPSHOTS

### 7.1 Comparative study of different models accuracy

Several performance metrics can be used to assess a model's effectiveness, including specificity, accuracy, recall, and precision.

#### Accuracy

Accuracy measures how well the model performs across all classes and is particularly useful when each class holds equal importance. It is calculated as the ratio of correctly predicted instances to the total number of predictions made. However, accuracy alone can sometimes be misleading especially when dealing with imbalanced datasets. In multiclass classification problems, accuracy simply represents the proportion of items that have been correctly classified.

#### Recall

Recall represents the proportion of actual positive instances that the model correctly identifies as positive, relative to all true positive samples. A higher recall indicates that the model successfully detects more positive cases. It remains unaffected by the number of incorrect classifications of negative samples. When a model accurately classifies every positive instance as positive, the recall value reaches 1.

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative}$$

## **Precision**

Precision measures the ratio of correctly predicted positive cases to the total predicted positive cases (both correct and incorrect). It indicates how reliably the model identifies an instance as positive. If the model produces many incorrect positive predictions, the denominator increases, leading to lower precision. Conversely, when the model makes more correct positive predictions (maximizing True Positives) and fewer incorrect ones (minimizing False Positives), the precision value is higher. Precision is therefore useful for understanding how accurate the model is when it classifies an instance as positive.

## **7.2 Front-end Results**

### **1. Dataset:**

The dataset used for the Intrusion Detection System (IDS) with Machine Learning was obtained from the NSL-KDD dataset, a refined version of the original KDD Cup 1999 dataset. It contains diverse network traffic records categorized as normal or various attack types, including DoS, Probe, R2L, and U2R. The collected data was organized, cleaned, and preprocessed to remove redundancies and inconsistencies, then stored locally for efficient access and integration during the model's training and testing phases.

### **2. Front-End:**

#### **2.1 Constraints Required:**

- Input format: Network traffic data in .csv or .pcap format (packet capture or structured dataset).
- File size: Up to 100 MB per upload to ensure efficient processing and analysis.
- Data content: Must include essential network attributes such as source/destination IP, ports, protocol, and payload size.
- Upload limit: One dataset file at a time for streamlined preprocessing and model evaluation.
- Response time: 3–5 seconds for preprocessing and initial prediction, depending on data size.
- **Browser compatibility:** Works on Chrome, Edge, and Firefox with JavaScript enabled.
- Internet requirement: Stable connection for API interaction between frontend and Flask backend

### 3. Web application snapshots:

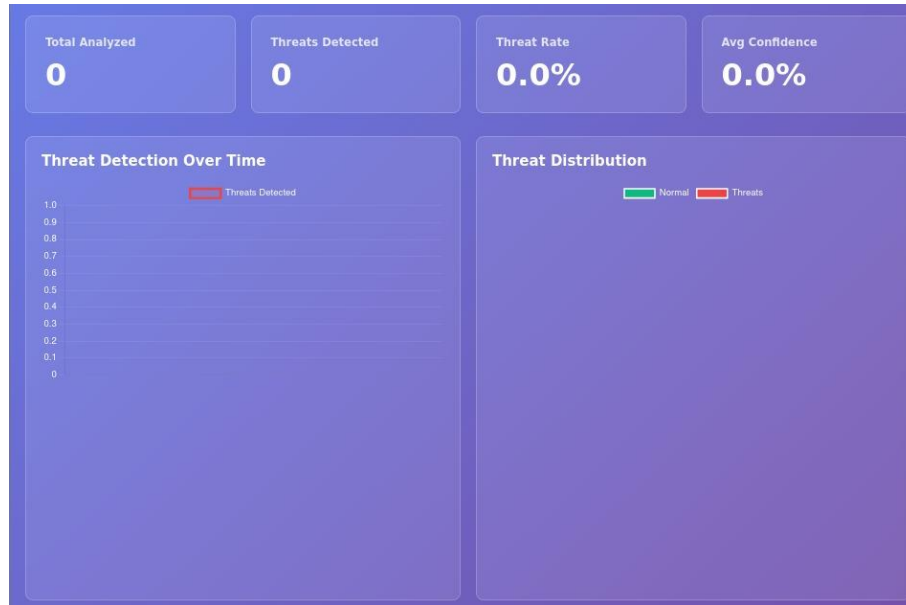


Figure 7.1- Interface of the Admin Dashboard

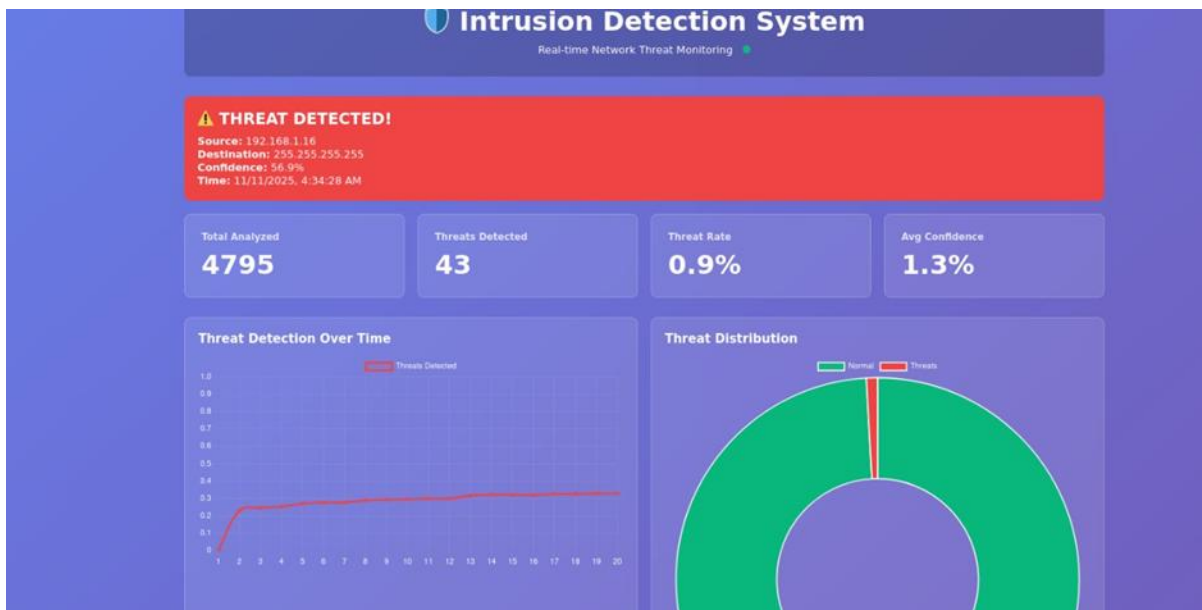


Figure 7.2- Interface when threat is detected

| Time        | Source IP       | Destination IP  | Status | Confidence |
|-------------|-----------------|-----------------|--------|------------|
| 12:44:06 AM | 192.168.11.147  | 103.167.211.146 | Normal | 14.0%      |
| 12:44:06 AM | 192.168.12.51   | 239.255.255.250 | Normal | 23.3%      |
| 12:44:06 AM | 192.168.11.147  | 103.167.211.146 | Normal | 14.0%      |
| 12:44:06 AM | 192.168.12.51   | 239.255.255.250 | Normal | 23.3%      |
| 12:44:04 AM | 192.168.1.16    | 255.255.255.255 | THREAT | 51.9%      |
| 12:44:06 AM | 103.167.211.146 | 192.168.11.147  | Normal | 22.7%      |
| 12:44:05 AM | 103.167.211.146 | 192.168.11.147  | Normal | 22.7%      |
| 12:44:05 AM | 192.168.11.147  | 103.167.211.146 | Normal | 14.0%      |
| 12:44:05 AM | 192.168.12.51   | 239.255.255.250 | Normal | 23.3%      |
| 12:44:05 AM | 103.167.211.146 | 192.168.11.147  | Normal | 22.7%      |
| 12:44:05 AM | 192.168.11.147  | 103.167.211.146 | Normal | 14.0%      |
| 12:44:05 AM | 103.167.211.146 | 192.168.11.147  | Normal | 22.7%      |
| 12:43:58 AM | 192.168.1.218   | 192.168.255.255 | Normal | 32.3%      |
| 12:44:05 AM | 192.168.12.51   | 239.255.255.250 | Normal | 23.3%      |
| 12:44:05 AM | 103.167.211.146 | 192.168.11.147  | Normal | 22.7%      |
| 12:44:05 AM | 103.167.211.146 | 192.168.11.147  | Normal | 22.7%      |
| 12:44:05 AM | 192.168.12.51   | 239.255.255.250 | Normal | 23.3%      |
| 12:44:04 AM | 192.168.11.147  | 103.167.211.146 | Normal | 14.0%      |

**Figure 7.3- Live data Capturing**

**Chapter – 8**

**CONCLUSION AND  
FUTURE WORK**

## CHAPTER 8

### CONCLUSION AND FUTURE WORK

The project on **Intrusion Detection System (IDS) using Machine Learning** highlights a system designed to monitor network traffic, detect malicious activity, and generate intelligent alerts in real time. By leveraging machine learning algorithms such as **Random Forest** and **Neural Networks** for traffic classification, the system efficiently distinguishes between normal and attack patterns.

The IDS effectively processes incoming network packets, extracts relevant features, and classifies them into various categories (such as normal, DoS, probe, or R2L attacks). The machine learning-based prediction layer enables adaptive and data-driven detection beyond traditional rule-based methods, while the alert module ensures immediate reporting of suspicious activities to the administrator. Overall, the system demonstrates a robust and scalable approach to securing network environments through intelligent anomaly detection.

For future work, several directions for enhancement and expansion are proposed:

- **Dataset Expansion:** Incorporate larger and more diverse network traffic datasets such as UNSW-NB15, CICIDS2017, or custom real-world captures to improve model generalization across various attack types and environments.
- **Real-Time Network Monitoring:** Integrate real-time packet capturing and streaming analytics capabilities for live detection and response, allowing the IDS to operate efficiently in dynamic and large-scale networks.
- **Adaptive and Self-Learning System:** Implement continuous learning mechanisms where the IDS retrains itself periodically using recent data, enabling adaptation to evolving cyber threats and attack strategies.
- **User Dashboard and Visualization:** Develop an interactive dashboard that visualizes network status, attack trends, and alert statistics, improving interpretability and decision-making for system administrators.

## **REFERENCES**

- [1] “A review on machine learning methods for intrusion detection system,” *Applied and Computational Engineering*, vol. 27, no. 1, pp. 57-64, Dec. 2023
- [2] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, “Survey of intrusion detection systems: Techniques, datasets and challenges,” *Cybersecurity*, vol. 2, no. 1, 2025. doi: 10.1186/s42400-019-0038-7
- [3] G. Kocher and G. Kumar, “Machine learning and deep learning methods for intrusion detection systems: recent developments and challenges,” *Soft Computing*, vol. 25, no. 15, pp. 9731–9763, 2021. doi: 10.1007/s00500-021-05893-0
- [4] A. J. A. Immastephy and K. Punitha, “A Systematic Review on Network Intrusion Detection System based on machine learning and deep learning approach,” *E3S Web of Conferences*, vol. 540, p. 14006, 2024. doi: 10.1051/e3sconf/202454014006
- [5] R. Kimanzi, P. Kimanga, D. Cherori, and P. K. Gikunda, “Deep Learning Algorithms Used in Intrusion Detection Systems - A Review,” *arXivpreprint arXiv:2402.17020*, 2024. doi: 10.48550/arXiv.2402.17020
- [6] S. M. Kasongo, “A Deep Learning Technique for Intrusion Detection System Using a Recurrent Neural Networks based Framework,” *Computer Communications*, vol. 199, pp. 113-125, 2023. doi:10.1016/j.comcom.2022.12.010
- [7] Y. Fu, Y. Du, Z. Cao, Q. Li, and W. Xiang, “A deep learning model for network intrusion detection with imbalanced data,” *Electronics*, vol. 11, no. 6, p. 898, 2022. doi: 10.3390/electronics11060898
- [8] A. Belenguer, J. Navaridas, and J. A. Pascual, “A review of Federated Learning in Intrusion Detection Systems for IoT,” *arXiv preprint arXiv:2204.12443*, 2022. (Submitted to IEEE Internet of Things Journal). doi: 10.48550/arXiv.2204.12443
- [9] P. L. S. Jayalaxmi, R. Saha, G. Kumar, M. Conti, and T. -H. Kim, “Machine and deep learning solutions for intrusion detection and prevention in iots: A survey,” *IEEE Access*, vol. 10, pp. 121173-121192, 2022. doi: 10.1109/ACCESS.2022.3220622
- [10] K. Mahendar and G. Shivakanth, “A Survey of Intrusion Detection Systems Based on Machine Learning for Cloud Security,” *SSRG International Journal of Electrical and Electronics Engineering*, vol. 12, no. 5, pp. 226-242, 2025. doi: 10.14445/23488379/IJEEE-V12I5P119
- [11] M. A. Umer, K. N. Junejo, M. T. Jilani, and A. P. Mathur, “Machine learning for intrusion detection in industrial control systems: Applications, challenges, and

- recommendations,” *International Journal of Critical Infrastructure Protection*, vol. 38, p. 100516, 2022. doi: 10.1016/j.ijcip.2022.100516
- [12] M. Mohale and I. C. Obagbuwa, “A systematic review on the integration of explainable artificial intelligence in intrusion detection systems to enhancing transparency and interpretability in cybersecurity,” *Frontiers in Artificial Intelligence*, vol. 7, 2025. doi: 10.3389/frai.2025.1526221
- [13] S. B. Ahamed Maricar, A. Anoop, B. E. Samuel, A. Appukuttan, and K. H. Alsinjlawi, “An Improved Explainable Artificial Intelligence for Intrusion Detection System,” *International Journal of Intelligent Systems and Applications in Engineering*, vol. 12, no. 14s, pp. 108–115, 2024
- [14] B. Mahbooba, M. Timilsina, R. Sahal, and M. Serrano, “Explainable artificial intelligence (XAI) to enhance trust management in Intrusion Detection systems using decision tree model,” *Complexity*, vol. 2021, Art. ID 6634811, 11 pages, 2021. doi: 10.1155/2021/6634811
- [15] Z. A. El Houda, B. Brik, and L. Khoukhi, ““Why should I trust your IDS?”: An explainable deep learning framework for Intrusion Detection systems in internet of things networks,” *IEEE Open Journal of the Communications Society*, vol. 3, pp. 1164–1176, 2022. doi: 10.1109/ojcoms.2022.3188750
- [16] O. Al-Badarin et al, “Adversarial Machine Learning Attacks against Intrusion Detection Systems: A Survey on Strategies and Defense,” *Sensors*, vol. 23, no. 4, p. 2075, Feb. 2023. doi: 10.3390/s23042075
- [17] Y. Senoussi and L. Salmi, “Transformer-Based Approach for Intrusion Detection System,” in *2023 International Conference on Digital Age and Contemporary Information Systems (DACIS)*, Sousse, Tunisia, 2023, pp. 1–6. doi: 10.1109/DACIS58652.2023.10200151
- [18] M. Zhong, M. Lin, and Z. Xu, “A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges,” *Computers & Security*, vol. 139, p. 103728, Mar. 2024. doi: 10.1016/j.cose.2024.103728
- [19] M. Driss, S. P. A. K. Al-Kassem, and Y. Jararweh, “Federated Machine Learning to Enable Intrusion Detection Systems in IoT Networks,” *Applied Sciences*, vol. 14, no. 6, p. 1176, Mar. 2024. doi: 10.3390/app14061176
- [20] H. Y. I. Khalid and N. B. I. Aldabagh, “A Survey on the Latest Intrusion Detection Datasets for Software Defined Networking Environments,” *Engineering, Technology*

